Design log: a simulated timetable feed

22 October 2026 · COMP1110 Group 08, Empty Classroom Plan

1. Why

The timetable belongs to the University. Our system must read it, but we have no access yet, and we should not touch the University's systems while we build.

So instead of connecting to their database or waiting for permission, I defined the interface we expect and built a stand-in that provides it. This has minimal impact on the University system (we only read, and never depend on it being up), and it means our own system can be actually built and run, not just designed on paper.

When real access arrives, we change one base URL in our adapter.

2. What I built

• A separate Spring Boot service on port 9000, with its own database (mockschool). No shared code with the main system.

• Three tables: rooms, class slots (weekly, Monday = 1), and feed metadata (generated_at, revision). Seed data exported from our own database: 10 rooms, 79 class slots.

• A read only API: /v1/meta, /v1/rooms, /v1/timetable?date=, /v1/timetable/{roomCode}?date=. All need X-Api-Key.

• A write API for rooms and slots. Overlapping classes are rejected here, because that rule belongs to the source of the timetable.

• Three test switches (stale, slow, down) and a browser control panel with a timetable grid and editable rooms and slots.

Ten checks pass: read endpoints, missing key (401), add a slot and read it back, overlapping slot (400), stale, slow, down, reset.

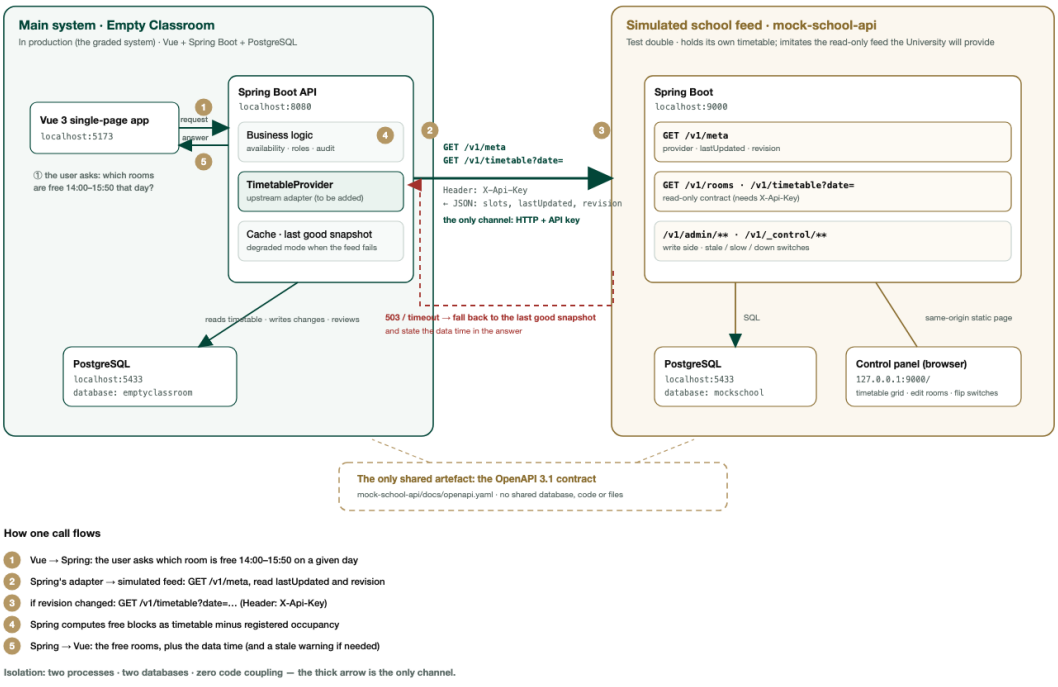


Figure 1. The main system and the simulated feed as two separate services. The numbers match the table below.

How the other system calls the feed

Step	What happens	Endpoint
1	The page asks our Spring Boot API which rooms are free in a time window.	Vue :5173 → Spring :8080
2	Our adapter reads the feed metadata: when the data was produced, and which revision it is.	GET /v1/meta
3	If the revision changed, it fetches the timetable for the requested date.	GET /v1/timetable?date=
4	Our service subtracts registered occupancy from the timetable to get free blocks.	in-process
5	The page receives the free rooms together with the data time.	Spring :8080 → Vue :5173

3. API documentation

• docs/API.md: every endpoint, request and response examples, error codes, and how to use lastUpdated and revision.

• docs/openapi.yaml: OpenAPI 3.1, 14 paths, 7 schemas. Usable in Postman and in a contract test.

The OpenAPI file is the only thing the two systems share. No shared database, code or files.

4. Problems and handling

Problem	Handling
The school updates the timetable	Poll on a schedule. Check /v1/meta every few minutes; fetch the full timetable only when revision changes. About 19 KB of metadata per day.
The school renames a field	Contract test in CI against openapi.yaml. Unknown fields are ignored; missing required fields are an error.
The data looks wrong	Sanity checks and a diff against the previous revision. A large change is not applied automatically.
The school system is down	Timeout, then use the last snapshot and state its time.
We might show old data as current	lastUpdated is shown in every answer.

A change in the meaning of a field cannot be detected automatically. Only a diff and a person catch that.

5. Next step

Add the adapter to the main system, with the polling, the four guards, a data time in the interface, and four tests.